

College Management System — Project Report

Prepared: 2026-06-15

Abstract

This document explains the current state of the "College Management System" repository, the changes made during the recent modernization effort, build and deployment instructions, test and CI details, and remaining tasks required for a hardened production release.

Keywords: Java, Spring Boot, MongoDB, Docker, CI, Maven, JaCoCo, Actuator, Deployment, Testcontainers, Docker Compose

1. Introduction

The College Management System is a console-based Spring Boot application that provides CRUD operations for college entities backed by MongoDB. The repository has been updated to be container-friendly, to enforce a minimum Java version, and to include a CI workflow.

This report summarizes the current repository contents, recent changes, and the exact steps to build, test, and deploy the application.

2. Project Goals

- Target Java 21 (minimum) for production builds and ensure CI uses the same runtime.
- Provide a reproducible Maven build and a CI pipeline that runs tests and builds a Docker image.
- Make the console UI optional so the app can run headless in production.
- Containerize the app, provide a `docker-compose.yml` for local integration testing, and make the Dockerfile robust across version changes.

3. High-level Architecture

Layered architecture:

- Console (conditional) → Service layer → Repository (MongoDB)
- Actuator / HTTP endpoints for monitoring

The production profile disables the console so the app starts in headless mode in containers.

4. Key Files and Components (changes highlighted)

- `pom.xml` — build file
 - `java.version = 21` (minimum). Added the Maven Enforcer plugin to require Java 21 or newer (`[21,)`).
 - JaCoCo configured with a compatibility profile (disabled on JDK `>= 22`).
- `Dockerfile` — multi-stage build updated to:
 - copy the built JAR using a wildcard (`/workspace/target/*college-management-system*.jar`) to tolerate versioned artifact names,
 - use `exec` in `ENTRYPOINT` so the JVM receives signals (graceful shutdown).
- `.dockerignore` — added to avoid sending build artifacts and IDE files to Docker context.
- `.github/workflows/maven-ci.yml` — CI workflow added. Builds with JDK 21, runs tests, packages the app, and builds the Docker image.
- `docker-compose.yml` — local compose stack added (MongoDB + app) that sets `MONGODB_URI` for local integration testing.
- `src/main/resources/application-prod.yml` — production profile (reads `MONGODB_URI`, disables interactive console, configures actuator exposure).
- `target/college-management-system-0.0.1-SNAPSHOT.jar` — packaged artifact (present in the repository `target/` folder from prior build runs).

5. Data Model

Student, Staff, Department, Book POJOs persisted via Spring Data MongoDB.

6. Implementation Notes

- The interactive console (`ConsoleMenu`) is conditional on `app.console.enabled` so it is disabled in `prod`.
- Resources like `Scanner` are closed properly on shutdown.
- Actuator endpoints `health` and `info` are available when running with `--spring.profiles.active=prod`.

7. Build, Test and Quality (current state)

Prerequisites for local verification:

- Java JDK 21 installed and `JAVA_HOME` set (local build in this environment failed earlier with `JAVA_HOME` not set).
- Docker installed to build and run container images locally (Docker was not available in the environment used for verification).

Important commands:

```
# Example: set JAVA_HOME (PowerShell)
$env:JAVA_HOME = 'C:\Program Files\Java\jdk-21'
cd college-management-system
.\mvnw.cmd -DskipTests package
.\mvnw.cmd test
```

Docker (local):

```
docker build -t college-management-system:latest .
docker run -e MONGODB_URI="mongodb://host.docker.internal:27017/college_db" -p 8080:8080 college-management-system:latest
# or using compose (starts Mongo + app)
docker-compose up --build
```

Notes about the verification performed in this environment:

- The target/ directory already contains `college-management-system-0.0.1-SNAPSHOT.jar` (packaged artifact).
- A local `mvnw` run in this environment failed due to `JAVA_HOME` not being set; please set `JAVA_HOME` and re-run locally to fully validate builds.
- Docker is not installed in this environment, so I could not build and run the container here; the `Dockerfile` was validated by inspection and made more robust.

8. Continuous Integration (GitHub Actions)

The pipeline added at `.github/workflows/maven-ci.yml` performs:

1. Checkout source.
2. Set up JDK 21 (Temurin) on the runner.
3. Build with Maven and run tests (`mvn -B -DskipTests package` then `mvn test`).
4. Build a Docker image using `docker/build-push-action`.

To publish images, configure registry credentials in repository secrets and extend the workflow accordingly.

9. Security & CVE Scan

- A direct-dependency scan on the main dependencies showed no critical direct CVEs at the time of the report. Re-scan after dependency upgrades.
- Recommendations: store DB credentials in secrets, secure Actuator endpoints (Spring Security), and use a vulnerability scanner (Dependabot, Snyk) in CI.

10. Observability

- Actuator endpoints `health` and `info` are exposed in `prod` profile; consider securing them for production.

11. Limitations & Known Issues (current)

- Local Docker builds were not executed in this environment (Docker CLI not available). Please run `docker build` or `docker-compose up --build` on a machine with Docker to validate the image and runtime behavior.
- Local Maven wrapper runs require `JAVA_HOME` to be set to JDK 21 for full parity with CI.
- JaCoCo: disabled automatically for JDK ≥ 22 by profile to avoid instrumentation issues; CI runs on JDK 21 to collect coverage.
- Integration testing coverage is minimal; add Testcontainers-based integration tests for MongoDB.

12. Future Work / Recommendations

- Add Testcontainers integration tests and run them in CI before publishing images.
- Secure Actuator endpoints with Spring Security and store secrets in a secret manager.
- Add liveness/readiness probes and resource limits for orchestrated deployments.
- Configure CI to push images to a registry and create promotion pipelines (staging/production).

13. Appendix

13.1. Quick file map (updated)

- `pom.xml` — build configuration (Maven); contains Maven Enforcer rule for Java 21+.
- `mvnw`, `mvnw.cmd`, `.mvn/wrapper/` — Maven Wrapper
- `src/main/java/com/college/CollegeManagementApplication.java` — main class
- `src/main/java/com/college/menu/ConsoleMenu.java` — interactive console (conditionally enabled)
- `src/main/java/com/college/entity/` — domain entities
- `src/main/resources/application-prod.yml` — production configuration
- `Dockerfile` — multi-stage container build (robust jar copy and `exec` entrypoint)
- `.dockerignore` — excludes build artifacts and IDE files
- `docker-compose.yml` — local Mongo + app for integration testing
- `.github/workflows/maven-ci.yml` — CI pipeline
- `PROJECT_REPORT.md` — this file

13.2. Key build and run commands (repeated for convenience)

```
# Build jar (skip tests)
./mvnw -B -DskipTests package

# Run tests
./mvnw test

# Build Docker image
docker build -t college-management-system:latest .

# Run locally with Mongo on host
docker run -e MONGODB_URI="mongodb://host.docker.internal:27017/college_db" -p 8080:8080 college-management-system:latest

# Or with Docker Compose (starts Mongo + app)
docker-compose up --build
```

13.3. Environment variables

- `MONGODB_URI` — connection string for MongoDB (fallback: `mongodb://localhost:27017/college_db`).
 - `JAVA_OPTS` — passed to the JVM by the Docker ENTRYPOINT.
-

14. Conclusion

A robust `Dockerfile`, `.dockerignore`, CI pipeline (`.github/workflows/maven-ci.yml`), and `docker-compose.yml` were added. A Maven Enforcer rule ensures builds use Java 21 or newer, and JaCoCo is configured to avoid instrumentation issues on very new JDKs.

Remaining verification steps to complete before final production rollout:

1. Run full test suite locally (set `JAVA_HOME` to JDK 21 and run `./mvnw test`).
2. Build and run the Docker image locally (install Docker) or run `docker-compose up --build` to validate runtime behavior with MongoDB.
3. Add integration tests (Testcontainers) and secure actuator endpoints as required by your security policy.

If you want, I can now:

- - A. Run the Maven build & tests here (requires `JAVA_HOME` set),
- - B. Run the Docker build here (requires Docker availability),
- - C. Add Testcontainers-based integration tests and wire them into CI, or
- - D. Prepare a PR with these changes and open a Pull Request for review.

Tell me which option to proceed with and I'll continue.